

Fabric Data Agent Workshop Labs

Participant guide | October 2026

Build a manufacturing operations data agent, improve its business understanding, evaluate its answers, and extend it with a Lakehouse source.

This guide follows the Microsoft Fabric Data Agent Workshop's three labs. Questions and actions are provided in execution order. Each exercise gives an **Observation** (what to look at, and what happened when we ran it), the **Expected result** (reference values), and **What to conclude**. Wording, generated queries, visual formats, and latency can vary between runs.

Starting point: SetupDataAgentJumpstart has already run successfully. Do not reinstall the Jumpstart or rerun setup just to begin this guide.

Formats: [Markdown](#) | [Download the PDF](#)

Contents

- Before you start
- Lab 1: Creating and optimizing data agents
- Lab 2: Programmatic evaluation of data agents
- Lab 3: Adding multiple data sources
- Workshop completion checklist
- Troubleshooting
- Sources and license

Before you start

Confirm the prepared environment

1. Sign in to [Microsoft Fabric](#) with the account provided for the workshop, then open your assigned workspace.
2. Open the `getting-started-data-agents` folder.
3. Confirm that both a semantic model and a report exist for `ManufacturingOps` and `ManufacturingOpsAIReady`.
4. Confirm that the following six notebooks are present.

Notebook	Role in the workshop
<code>SetupDataAgentJumpstart</code>	Already completed: imports and configures the populated models and reports.
<code>RefreshSemanticModel</code>	Optional maintenance; not required simply because the workshop is starting.
<code>JudgeCalibration</code>	Calibrates and registers the evaluation judge in Lab 2.
<code>EvaluateDataAgent</code>	Evaluates your AI-ready agent in Lab 2.
<code>BuildOpsRefData</code>	Builds the Lakehouse and its SQL-facing objects in Lab 3.
<code>CreateMultiSourceDataAgent</code>	Configures and publishes your multi-source agent in Lab 3.

Observation: two semantic models, two reports, and six notebooks are available. The models contain cached data. Notebook presence alone does not prove that setup completed: open the two reports and confirm that their visuals load data rather than displaying connection errors.

Both reports open on a blank **Page 1**. Use the page tabs at the bottom:

- `ManufacturingOps` report: open the **info** page. It shows the data-as-of card.
- `ManufacturingOpsAIReady` report: open the **Verified Answer** page. It shows scrap rate by manufacturer.

If different: ask the facilitator to finish or repair setup. Do not create duplicate models, change capacity, or rerun setup over other participants' work. A successful refresh is not a prerequisite when the populated imported models already work.

Facilitator note — known setup failure: in the tested notebook version, the first `SetupDataAgentJumpstart` run can stop at the `DataFolder` parameter update with **HTTP 403**. The cause is one request whose `Authorization` header contains a literal placeholder instead of the token. In that cell, set the header to `"Bearer " + power_bi_token` (the notebook's existing Power BI token variable), or ask notebook Copilot to fix the error, then rerun. The second run completes both parameter updates and the final data checks. No permission change is needed.

Access and optional features

The facilitator normally prepares the paid Fabric capacity, Power BI Pro license, workspace Contributor access, and required Data Agent/Copilot tenant settings. If **New item > Data agent** is unavailable, stop and ask the facilitator to check these prerequisites.

Model editing and notebook execution require the appropriate permissions. Modeling Copilot, Preview runtime, Code Interpreter, and Microsoft 365 Copilot integration can have additional tenant, licensing, or regional

requirements. Their absence is a feature/access blocker, not evidence of an incorrect analytical answer. Record any section you cannot execute.

Use your own agent names

Choose a short, unique identifier, such as your initials followed by a number. This guide uses AB01 as an example. Replace it consistently with your own identifier; do not use angle brackets in the item names.

Item	Example name
Baseline agent	MfgOps_DA_AB01
AI-ready agent	MfgOps_DA_AIReady_AB01
Multi-source agent	MfgOps_DA_AIReady_AB01_MultiSource

In a shared workspace, model changes affect everyone using those models. Coordinate the model-editing exercises with the facilitator. Do not rename or overwrite somebody else's agent. For a resumed lab, reuse only your own items and inspect their configuration before continuing.

Know the business and the data

The lab uses a fictional industrial manufacturer. It makes about 20 products (pumps, turbines, motors, valves, sensors and spare parts) on 8 production lines in several plants, then sells them to business customers. You do not need a manufacturing background: the terms below cover everything the questions use.

Main tables

Table	One row is...	Used for
ProductionLog	one production run: a line, a shift, a day	planned, produced, good and scrap units; runtime and downtime minutes
Inventory	the stock of a product in a plant on a date (a daily snapshot)	stock on hand and reorder level
Products	a product (SKU)	name, category, list price, cost
Lines / Plants / Assets	a production line / a factory / a piece of equipment	where and on what equipment production runs; Lines[Manufacturer] is the equipment maker
Sales / Customers	an order line / a buying company	revenue and units sold
PurchaseOrders / Vendors	a purchase-order line / a supplier	purchasing spend and supplier delivery
Date	a calendar day	all time filters
Business Measures	no rows; it only holds the measures	the governed KPIs below

Key metrics and terms

Term in the questions	Meaning	Measure
Production quantity	units produced = good units + scrap units	[Production Qty]
Scrap rate (RQX)	share of produced units that were rejected	[Scrap Rate %]
Yield	share of produced units that were good (the opposite of scrap rate)	[Production Yield %]
Day production yield (DPY)	yield excluding the Night shift , not "yield per day"	[Day Yield Pct]
OEE ("reliability" in this lab)	Overall Equipment Effectiveness = Availability × Performance × Quality	[OEE %]
Availability / Performance / Quality	share of time running / produced vs planned / good vs produced	[Availability %], [Performance %], [Quality %]
Downtime	minutes a line was stopped	[Downtime Minutes]
Reorder level	stock level below which a product should be replenished	used by the inventory measures
Inventory risk count	number of daily product-plant snapshots below the reorder level	[Inventory Risk SKU Count]
Total sales / revenue	net sales amount in USD (not list price × quantity)	[Total Sales]
TP	the Pumps and Turbines product categories (in Lab 3: "turbomachinery")	[sls_amt_x] for TP sales
YoY	year over year: the same period one year earlier	—

Two models hold this data:

- ManufacturingOps (**baseline**): deliberately unfriendly, with cryptic names such as custName, prd_yld_day and sls_amt_x, and few descriptions. It shows what an agent does without business context.
- ManufacturingOpsAIReady: the same business with clear names, a description on every measure, and the AI preparation you complete in Lab 1.

In Lab 3 the OpsRefData Lakehouse adds two things the models do not have: the **reasons** for downtime and monthly product sales.

How to read a result

For each question, review three outputs: the **paraphrased question** (how the agent restated your request), the **generated DAX or SQL** with its result, and the **final answer**. Expand the run-step control beneath the answer ("1 step completed"); its label and number of steps can vary. The agent works from names, descriptions, Prep data for AI and instructions. It does **not** read DAX code or comments, so meaning hidden there is invisible to it.

An acceptable numerical answer uses the intended measure, entity, grouping, and period, and agrees with the query output. Fluent prose alone is not sufficient. A query error is not an empty dataset; an empty dataset is not numeric zero.

Use **Clear chat** between independent tests. Keep the same conversation only for exercises explicitly identified as follow-ups: the agent keeps conversation context, so phrases like "the top three" refer back to the previous answer. Clear chat asks for confirmation ("Clear chat? This will erase all chat history and start a new chat."); you can tick **Don't show this again**.

Each exercise below shows the results observed when this guide was rehearsed end to end, three times, on fresh deployments (3 October and twice on 4 October 2026). Where runs differed, the differences are shown: that variability is part of what you are learning to check.

- **Observation** tells you what to inspect and what the agent did in the rehearsals, including behavior that is deliberately imperfect on the baseline agent. That imperfection is the lesson of the exercise, not a broken lab. Run the **Recovery prompt** that follows it to see the correct result.
- **Expected result** gives the values from the workshop sample data. Your wording, layout and timing will vary; the measure, period and numbers should match.
- **What to conclude** is the takeaway to discuss before moving on.

Establish data-as-of dates before relative questions

Why this matters: "last month" can mean the previous calendar month relative to today, or the previous month relative to the latest loaded fact date. Those are different questions when the imported data is historical. A refresh timestamp tells you when a model was processed; it does not tell you the date of the latest business event.

After connecting each agent to its model in Lab 1, ask:

What are the earliest and latest dates with production records in this model?
Use ProductionLog dates, not the last date in the calendar table. Return one row with both dates. Do not apply a default time window.

1. Expand the run step and verify that the query reads ProductionLog dates, rather than only the calendar table.
2. Write down the minimum and maximum dates for each model separately.
3. Do not assume that the baseline and AI-ready models have identical data.
4. Do not assume that a maximum date proves every earlier day is complete. Inspect monthly counts and consult the facilitator if completeness matters.

Expected result (workshop sample data): the models are imported snapshots, so every deployment returns the same coverage:

Source	First production date	Latest production date
ManufacturingOps (baseline)	1 June 2024	6 July 2026

Source	First production date	Latest production date
ManufacturingOpsA IReady	1 June 2024	8 August 2026
OpsRefData Lakehouse (Lab 3)	built relative to the day you run BuildOpsRefData	runs up to that day

Neither model contains data up to today. This is why "last month" in the baseline returns **June 2026**, and why Lab 3 needs explicit dates when it combines the models with the Lakehouse.

Fact tables inside one model can also end on different days. In ManufacturingOpsAIReady, production runs to **8 August 2026** but the last inventory snapshot is **3 August 2026**, so "the latest 30 days" of inventory can be anchored on either date (see Step 4).

If you also ask for "the number of production records", the agent may return the count on each boundary date rather than the all-history total. That is an ambiguity in the question, not a data problem. Use the DAX check below for an exact count.

The check is **optional**. You do not need it for the later questions to work. It tells you whether an unexpected period comes from the agent's interpretation or from data that is not loaded. If the answer is unavailable or ambiguous, open the semantic model's **DAX query view**, create a query, and run this read-only check. If that view is not available to you, ask the facilitator to run it.

```
EVALUATE
ROW(
    "First production date", MIN(ProductionLog[Date]),
    "Latest production date", MAX(ProductionLog[Date]),
    "Production rows", COUNTROWS(ProductionLog)
)
```

For numerical comparisons between agents, choose the same explicit date range within both models' coverage. Use the relative questions as a separate test of time interpretation, rather than silently changing the question to fit the data. A model ending partway through a month should not be presented as a complete month.

Workshop time convention: when no period is supplied, the AI-ready agent will use the latest 30 calendar days **including** the latest production date: from that date minus 29 days through that date. Explicit user dates take precedence. Ambiguous relative phrases should be clarified or accompanied by the exact resolved dates. In production, the business owner must decide whether such phrases are anchored to today or to a data snapshot.

Facilitator cue: "Before trusting the number, we check which dates it describes. The agent cannot report a month that has not been loaded. We want to distinguish data freshness from query correctness."

Lab 1: Creating and optimizing data agents

Goal: build a baseline agent, diagnose ambiguous questions, configure an AI-ready model and agent, and explore runtime and analysis tools.

Step 1: Creating your first data agent

1. In the prepared folder, select **New item > Data agent**. Use the item search if needed.
2. Name the agent MfgOps_DA_AB01, using your own identifier. Take a moment to look at the interface: **Explorer** (Data, Setup, Tools tabs) on the left, the test chat in the centre, and the toolbar (**Add data**, **Add tools**, **Build agent with AI**, **Agent instructions**, **Runtime**, **Publish**).
3. In **Explorer**, select **Add data > Data source**.
4. In the OneLake catalog, select the ManufacturingOps **semantic model** in your workspace and select **Add**. Models with the same name from other workspaces can appear in the list, and the catalog does not show a workspace column (columns: Name, Type, Owner, Refreshed, Endorsement, Sensitivity). Use the **Refreshed** time (the time setup ran in your workspace) or the **Filter** to pick yours.
5. Expand the model and **select all its tables** for this baseline exercise. Selecting the model root shows a warning that the Standard runtime supports up to 25 tables; this model has 13, so select **Continue with Standard**. Confirm that each checkbox is selected; adding a source is not the same as selecting its tables.
6. Run the data-as-of check above. Record this model's production coverage (reference: 1 June 2024 to 6 July 2026; the generated DAX uses MIN/MAX of ProductionLog[Date]).
7. Clear chat and ask the introduction question.

I am new to this agent and the data. Tell me more about it and how to use it.

Observation: the agent lists the connected tables and measures, groups them by topic (production, inventory, purchasing, sales) and suggests example questions. Try one or two of its suggestions. Read it critically: in the rehearsals it said "instead of you writing SQL" (this source is queried in DAX), described Production Yield % as "often similar to prd_yld_day" (it is not: prd_yld_day excludes the Night shift), and offered sales and purchasing questions that a later, operations-only agent will refuse.

Expected result: a structured orientation (what data, what questions, how to phrase them) in about 20–50 seconds, with no query required.

What to conclude: The agent describes what it *thinks* it can do from table and column names. That is a starting point, not a list of verified capabilities: its guesses about meaning can already be wrong.

Ask the following one at a time. Clear chat between these independent tests.

Show me the production quantity for the last month.

Observation: check whether the DAX anchors "last month" to today or to the latest loaded date, and whether the answer says which month it used.

Expected result: 45,078 units for **June 2026** in all three rehearsals. The month was named in some runs ("the last full calendar month (June 2026)") and not in others ("the last calendar month is 45,078 units").

How to read it:

Check	Observation
What you meant	The previous calendar month before today (for example, September 2026 if you run the lab in October).

Check	Observation
What the agent did	Used the latest data in the model (data ends 6 July 2026) instead of today, so "last month" became June 2026.
Is the number wrong?	No: 45,078 is the correct June total.
Is the answer right?	No: it answers a different question. Even when it names June, it never says that the model has no data for the month you meant.

What to conclude: Both conventions ("before today" or "before the latest data") are legitimate, and the model does not say which one applies. The failure is not choosing June; it is not saying so. Relative dates are ambiguous. A trustworthy answer states the exact period used and when the data stops. The AI-ready agent is configured to do this later in the lab.

If different: ask the agent to state the exact dates. Repeat with an explicit month inside the recorded coverage. Do not describe "no records" as zero production, and do not silently relabel an older month as last month.

List the top 5 products by total sales.

Observation: check the ranking measure and the period. The baseline exercise includes sales; the operations-only agent created later will deliberately decline sales questions.

Expected result: HelioGen 3000 Steam Turbine (\$5,905,036,800), HelioGen 2000 Gas Turbine (\$4,585,175,100), HelioGen 1000 Gas Turbine (\$3,113,023,400), AquaFlow 200 Centrifugal Pump (\$856,623,040), AquaFlow 350 Submersible Pump (\$855,677,940), over **all available history**, in about 15 seconds. In one rehearsal the answer said "across all available data"; in the other two, the period appeared only in the paraphrase and DAX.

What to conclude: The ranking is right, but an answer without its period is incomplete: "top 5" over two years is not "top 5" this quarter. Always check that the period appears in the answer, not only in the query.

If different: inspect the ranking, measure, and period. Product list prices are not total sales. If the query returns fewer than five products, distinguish insufficient data from an omitted result.

Give me a pivot table with product, total sales, quantity, and inventory.

Observation: check whether "quantity" means units sold or produced, and whether inventory is one snapshot or a sum over many snapshots. A text table rather than an interactive pivot is normal.

Expected result (deliberately imperfect): a 20-product table with total sales, sales quantity and inventory over all available data, with no period stated. Inventory is **summed across every daily snapshot** (about 62,000–69,000 per product, for example AquaFlow 100 Centrifugal Pump 64,286), so it is not a stock level.

Recovery prompt (clear chat first):

For June 2026, give me a table with product, total sales and sales quantity.
Add inventory on hand from the single latest inventory snapshot in June 2026 only; do not sum inventory across dates. State the snapshot date.

Expected result: June sales with inventory from **one** snapshot, labelled **29 June 2026** (for example AquaFlow 100 Centrifugal Pump: 37,326,870 sales, 9,335 units, 285 on hand), in about 1 minute.

What to conclude: Stock is a snapshot: you can add sales across days, but not stock levels. The agent does not know which measures can be summed over time unless the model or the question tells it. A professional-looking table can still be meaningless.

Multi-part question and conversational context

Clear chat, then ask:

Which products have inventory below the reorder quantity? How often does that happen?

Observation: the agent may break a multi-part question into several sub-questions or answer it with a single query, and it may add a visual. Here, check **which unit** it uses for "how often": the model has a governed measure, Inventory Risk SKU Count, described as the number of product-plant-date snapshots below reorder. Frequency must not be confused with shortfall units.

Expected result (deliberately imperfect): the unit changes between runs. All 20 products were below reorder at least once, but the rehearsals counted differently:

- One run used the governed Inventory Risk SKU Count: the number of **product-plant-date snapshots** below reorder (FlowGuard 10 Control Valve 41, TorqueMax 30 Induction Motor 39).
- Two runs wrote their own DISTINCTCOUNT of **dates** below reorder (FlowGuard 10 Control Valve 38 dates, FlowGuard 25 and TorqueMax 30 37 each), once with a percentage of 220 dates.

What to conclude: "How often" needs a defined unit. Even with a governed measure available, the agent does not always use it, so the same question can return different numbers. Read the stated unit, not just the figures. Here the right unit is the governed one, **product-plant-day snapshots**, because stock is recorded per product, per plant, per day (FlowGuard 10: 41 snapshots vs 38 distinct days, since on 3 days both plants were short). You will fix that definition in Step 3 and check it in Step 4.

Without clearing chat, ask:

For the top three products by frequency, show me the monthly trend as a bar chart.

Observation: the follow-up should refer to the same three products and the same definition of frequency. The agent identifies "the top three" from the previous answer; it can use one query or several (the original lab saw three separate queries combined into one visual). Compare the chart with the table in the run step.

Expected result (deliberately imperfect): in all three rehearsals the **chart showed a single bar** (the first month, June 2024) although the query returned every month (the run step shows the full monthly table). In one run the follow-up also **switched the metric** from snapshot counts to distinct days; in the third rehearsal it kept the "dates" unit of the previous answer (FlowGuard 10, FlowGuard 25, TorqueMax 30).

Recovery prompt (same conversation; it names the measure explicitly, so it works whatever unit the first answer used):

Use the governed [Inventory Risk SKU Count] measure: the number of product-plant-date inventory snapshots below reorder quantity. Take the top three products by that measure across all history, breaking ties by product name. Return a table with product, month and that count for every month with data.

Expected result: a product-by-month table for AquaFlow 100 Centrifugal Pump, FlowGuard 10 Control Valve and TorqueMax 30 Induction Motor (FlowGuard 10 = 41, TorqueMax 30 = 39, AquaFlow 100 = 38, first alphabetically among four products tied at 38). Months without a risk snapshot are omitted. The monthly counts add up to each product's total. Prefer the table to the chart.

What to conclude: A follow-up carries over *which* products you discussed, not necessarily *how* you measured them. In multi-turn analysis, restate the metric, the tie-break and the period, and check a chart against its underlying table.

If different: confirm the three product names, date range, and frequency definition explicitly, then retry. If the previous answer did not establish a valid ranking, there is no trustworthy "top three" to carry forward.

Domain-specific KPI question

Clear chat and ask:

For the bottom 20% of products by revenue, do we run into inventory issues? Why?

Observation: expand "1 step completed" and inspect the DAX. Without any instructions, the agent interprets the vague "inventory issues" from the schema: it **reuses existing measures** (Total Sales, Total Inventory Qty, Inventory Below Reorder Qty, Inventory Risk SKU Count) and **builds new calculations** on top of them (ratios, per-product averages) inside the query. Check how it defines "bottom 20%" and ties, and whether each conclusion is supported by a number.

Expected result (deliberately imperfect): the bottom 20% is correctly identified (4 products, \$202.5 million revenue), but the answer **goes beyond the data**:

- In two rehearsals it invented causes: "lower demand", "conservative stocking policies", "reorder points set too low", "lumpy spare-part demand", "prioritization bias", once under a heading called "Data-grounded reasoning".
- In the third, it compared the bottom 20% with the other products (2.76% vs 2.99% of units below reorder; 31.25 vs 34.7 risk snapshots per product, mislabelled "SKUs") and concluded that the real issue was **overstock** (25× more inventory per sales dollar). That figure divides **summed daily snapshots** by revenue, the same error as the pivot table, and it then suggested "over-forecasting, long lifecycles, or minimum-order quantities" as causes.

Recovery prompt (clear chat first):

List the products in the bottom 20% by total revenue across all history, with their revenue and their count of inventory snapshots below reorder quantity. Report only what the data shows. If the data cannot explain why, say so instead of suggesting causes.

Expected result: four low-revenue products: Pump Seal Kit (\$23,424,463.50, 30), Pump Impeller (\$42,591,626, 37), SenseLine Temperature Sensor (\$66,018,416, 30), Turbine Bearing Set (\$70,501,664, 28), and an explicit statement that the data does not explain why.

What to conclude: Asking "why" invites plausible-sounding explanations that the data cannot support, and an improvised calculation can turn a modelling mistake (summing stock) into a confident business finding. Ask for facts, ask the agent to say when the data cannot explain a cause, and keep evidence and hypotheses separate.

If different: request the revenue period, selected products, and inventory criterion. "Why" does not authorize a causal explanation absent supporting data. Distinguish measured associations from hypotheses.

Facilitator cue: "A plausible answer is the start of the review, not the end. We inspect the measure, time window, and business definition."

Step 2: Improving and optimizing data agent responses

The baseline agent produced natural-language answers grounded in the model. To improve them, you first need to see how it reaches an answer and where it has to guess. Stay in the baseline agent. Clear chat before each question, expand its run steps, and compare the **paraphrased question**, the **generated DAX**, and the **final answer**. These are diagnostic exercises: the baseline is not required to fail in a particular way.

Q1: Scrap rate and implicit dates

What is our scrap %?

Observation: the agent finds the existing [Scrap Rate %] measure but has to guess the time window: all history, the latest date, or a recent window. The answer is not wrong, but the assumed period is not guaranteed

to match what the user meant.

Expected result: 2.35% from ROW([Scrap Rate %]) over **all available history**, with "across all available data" in the answer, in about 10 seconds (identical in all three rehearsals).

What to conclude: When a measure is clearly named (Scrap Rate %), even the baseline uses it correctly. What is still missing is an agreed default period. Instructions can steer that: the AI-ready model adds a default of the latest 30 days.

Q2: A named KPI without a governed definition

What is our OEE?

Observation: OEE is not defined anywhere in the baseline model or agent, although it is a standard manufacturing term. The agent infers "Overall Equipment Effectiveness" from the schema and builds the KPI inside the query from other measures. Expand the run step to see the improvised definition.

Expected result (deliberately imperfect): an ad hoc formula, not a governed measure. Two rehearsals returned **87.33%** over all data (Availability 96.86% × Performance 92.33% × Quality 97.65%, where Performance is Schedule Attainment % and Quality is Production Yield %); another returned **92.6%** with the Performance factor missing and no date stated.

What to conclude: Without a governed definition, the agent invents a formula, and it can choose a different one the next time. The result looks credible, but it is not *your* OEE. Rule of thumb: any KPI users ask for by name (OEE, scrap rate, yield) should be a governed measure in the semantic model, so the agent reuses it instead of improvising.

Next action: the AI-ready model contains the governed [OEE %].

Q3: Day-shift yield versus daily yield

What is our day production yield for the last six months?

Observation: "day production yield" here means yield **excluding the Night shift**, implemented by the poorly named [prd_yld_day] measure. Its meaning sits only in a DAX comment, which the agent does not read. Check whether it uses that measure or simply groups all-shift yield by calendar day, and check the dates it computed for "six months".

Expected result (deliberately imperfect): in all three rehearsals the agent used **all-shift** Production Yield % per day or per month (about 97–98%), not the day-shift measure. In the third run, the DAX comment said "last six calendar months" but the code (EOMONTH(max, -5) + 1) started on 1 March, so the result covered only **2 March – 6 July 2026** (109 daily rows). In another run the agent claimed the model "only has data for these five months", which is false: data starts in June 2024.

What to conclude: The correct measure exists, but its name does not say what it means and the agent cannot read DAX comments, so it never finds it. Business meaning must be written where the agent looks: names, descriptions, Prep data for AI and instructions. And check the dates in the query, not the comment beside them.

Next action: the AI-ready model uses [Day Yield Pct] with a description and an AI instruction. If the period extends outside coverage, the answer should disclose that rather than silently shortening it.

Q4: Internal acronyms

What were the TP sales last week?

Observation: nothing in the model's names says what "TP" means. At this company it is the **Pumps and Turbines** categories, computed by the cryptically named [sls_amt_x]. Aliases and acronyms like this are common and an agent has no way of knowing them.

Expected result (deliberately imperfect): three different failures in three rehearsals:

- It searched product names for "TP", got **BLANK** and reported **0 sales**.
- It said no TP designation exists, then reported **total sales for all products** for the last week (254,725,180), answering a different question.
- It filtered product codes starting with "TP" for the last week of the *calendar table* (using week number only), got an empty result, listed possible reasons (including "no sales last week") and **asked how TP is identified**.

What to conclude: Internal jargon is guessed, and the guess is often presented as an answer: a confident "0" from an empty result, or a substitute total. The third behavior, asking for clarification, is the one you want, and you can request it in the instructions. Names, descriptions and synonyms are the primary context for acronyms and internal codes.

Next action: the AI-ready agent knows TP but must decline this **sales** request: recognizing a term does not put it in scope.

Q5: Business meaning of "machine"

Show me the distribution of scrap rate by machine.

Observation: "machine" can mean an asset, a production line or an equipment manufacturer. Inspect the grouping column in the DAX, not just the chart label.

Expected result (deliberately imperfect): the baseline groups by **asset** (`Assets[Name]`) over all history: eight assets between 2.28% (Coil Winder B2) and 2.42% (Feed Pump B1). In the third rehearsal the agent noticed the inactive `Assets–ProductionLog` relationship and activated it with `USERELATIONSHIP` in its own measure; in another, three assets showed no value and the chart showed one category only. The chart's axis may not start at zero, which exaggerates small differences.

What to conclude: "Machine" means different things to different people. The agent picks one; the business must define which. A column name can mean one thing in the schema and another to the business ("Customer" may really be a user, "Product ID" a SKU). Verify the grouping in the query and table, not the chart.

Next action: for this workshop's AI-ready exercise, "machine" is a business alias for **equipment manufacturer**, mapped to `Lines[Manufacturer]`. This is a workshop-specific convention, not a universal meaning of machine. Use `Lines[Manufacturer]` because production rows reach it through an **active** relationship; the `Assets` relationships to `ProductionLog` and `Lines` are **inactive**, so grouping a production KPI by an `Assets` column does not filter it. The bundled verified answer also uses `Lines[Manufacturer]`.

Facilitator cue: "We are not teaching the agent a universal definition of machine. We are documenting the meaning our business expects."

Step 3: Configuring and optimizing the semantic model and data agent

A trusted data agent comes from analysing its answers and configuring it deliberately. Start by fixing its scope, then use the optimization controls.

Define scope

A data agent can use several sources, so first decide its scope: a topic (for example inventory management), a domain (manufacturing operations) or an area (a single plant or product line). The next agent answers manufacturing operations questions: production, inventory, assets, plants, lines, scrap, yield, and OEE. It does not answer sales, customer, vendor, or purchasing questions. Lab 3 will explicitly add product sales through a different source.

Leave unnecessary tables, columns and measures out of the agent's scope, but do not remove sales tables from the underlying shared model to enforce this exercise. Focus the AI schema and agent table selection, and add scope instructions. These controls are not a replacement for permissions or row-/object-level security.

Prepare the semantic model for AI

People and AI use data differently: AI needs the business context people carry in their heads. There are four groups of controls:

Control	What it covers
Semantic model foundation	star schema, relationships, RLS/CLS, schema names
Business logic	governed measures and columns
Semantic metadata	descriptions, synonyms, hierarchies
AI readiness and context	Prep data for AI (AI data schema, verified answers, AI instructions) and the data agent configuration

Inspect the AI-ready model

1. Return to the workspace and open the ManufacturingOpsAIReady **semantic model**, not its report.
2. Switch to **Editing** mode when authorized.
3. Inspect names and descriptions. Examples include Customers[Customer Name], Assets[Asset Name], [Day Yield Pct], [Scrap Rate %], and [OEE %].
4. Confirm that descriptions explain business meaning and usage, rather than merely repeating the field name. Customers[Customer Name], for example, carries an example value and usage guidance that separate it from asset or product names.
5. Inspect [sls_amt_x] as a metadata example: the description explains TP, although sales will remain outside this agent's scope.

Observation: every table, column and measure has a business-friendly name or a description written for AI retrieval. Not every object is renamed: [sls_amt_x] keeps its cryptic name because downstream items depend on it, and the description carries the meaning instead.

What to conclude: When renaming would break reports or code, a good description is the next best fix: it is what the agent reads.

Check Prep data for AI

1. In the semantic model, select **Prep data for AI**. It optimizes the model for Copilot and data agents through three settings.
2. Open **Simplify the data schema / AI data schema**: a focused subset of the model that Copilot and data agents prioritize. The deployment has **already** focused it: the operations tables Assets, Business Measures, Date, Inventory, Lines, Plants, ProductionLog, and Products are included, while Customers, PurchaseOrders, Sales, SalesSummary, Vendors and the sales/purchasing measures are excluded. Confirm this rather than changing it. The agent's Explorer still lists all 13 tables: in Step 4 you select the same eight tables in the agent itself.
3. Review **Verified answers**: human-approved visual answers with trigger phrases and optional filters, which improve accuracy and consistency. Look for the scrap-rate-by-machine example and inspect its grouping,

measure, and filters.

4. Confirm that the verified answer groups by Lines[Manufacturer]. This matches the rule below. Do not switch it to Assets[Manufacturer]: the Assets relationships are inactive, so that column does not filter production measures.
5. Open **Add AI instructions**: business logic and terminology written on the model, which influence the DAX the agent generates. The bundled text conflicts with this workshop in two places: RQX = [Quality %] measure (RQX is scrap rate) and For all questions related to "machines" use Assets[Manufacturer] column (that column does not filter production measures). **Select all the existing text and replace it** with the block below. The block keeps the bundled rules that remain valid, including the CONTAINSSTRING rule for names, and adds the definition of "how often" from Step 1. It is the instruction set used in the third rehearsal (lightly reformatted for reading).

Use governed measures rather than inventing alternative KPI formulas.

For production questions with no period, use the 30 calendar days ending on the latest ProductionLog[Date], inclusive: latest date minus 29 days through latest date. Do not use the maximum calendar-table date as the data-as-of date.

Explicit user dates take precedence. For ambiguous relative phrases such as "last month", "last week", or "this year", establish the reference date and calendar convention. Return the exact start and end dates used. Disclose incomplete coverage; do not substitute an older period without saying so.

User-specified durations are explicit periods and ALWAYS override the default: six weeks is 42 calendar days, ending on the latest production date unless the user specifies another reference. Use end minus 41 days through end, inclusive, never 30 days for a six-week question.

For every production KPI query, include the actual latest ProductionLog date as a Data As Of result alongside the requested start and end dates. A requested end date later than Data As Of does not mean those later dates contain data.

Day production yield / DPY means yield excluding the Night shift. Use [Day Yield Pct], not all-shift yield grouped by day. Day production yield is a day-shift-only KPI, not a request for a daily time series. When asked to break a KPI down by line for a period, evaluate the governed measure over the entire period and return one row per line. Add daily grouping only when explicitly requested. Do not average daily percentages.

RQX means [Scrap Rate %], not [Quality %].
OEE means [OEE %]. In this workshop, "reliability" is an alias for OEE.
TP refers to Products[Category] values Pumps and Turbines, not Motors.

In this workshop, machine means equipment manufacturer. For a breakdown by machine, group ONLY by Lines[Manufacturer]; do not add asset name, line or location unless the user explicitly asks for that detail. For individual equipment, use Assets[Asset Name]. Use Lines[Manufacturer] through the active Lines-to-ProductionLog relationship: the Assets-to-ProductionLog and Assets-to-Lines relationships are inactive, so grouping a production KPI by Assets attributes does not filter that KPI.

Aggregate at the requested grain before returning results. Do not derive a full-period total or average from a truncated detail result.

Unless the user requests missing groups, exclude rows with BLANK production quantity and BLANK KPI. Preserve actual numeric zeros. Never replace BLANK with zero, including for visualization.

"How often" for inventory below reorder means [Inventory Risk SKU Count]: the number of product-plant-date snapshots below reorder quantity. State that unit.

Never calculate sales revenue from Products[Price].

If a query fails, report an execution problem. Do not claim that data is missing unless a successful coverage query supports that statement.

For named-entity columns (e.g., names, places, organizations), use `CONTAINSSTRING` for partial-text matching by default. Use exact-match filters only when the user explicitly requests a specific entity.

6. Search the saved instructions for `Quality %` and `Assets[Manufacturer]`. Neither should appear as a rule (only "not [Quality %]" remains).

7. Save/apply the model changes and close the Prep data for AI pane.

Observation: the AI schema, verified answer and instructions now express one consistent business interpretation (rehearsal 3: the saved instructions contained neither `RQX = [Quality %]` nor the `Assets[Manufacturer]` rule).

Important: two instruction layers exist. Model AI instructions in **Prep data for AI** shape the DAX the agent generates ("always calculate this way"). Agent instructions shape orchestration, scope, routing and response format ("always respond this way"). Neither substitutes for the other, as Step 4 shows.

Verified answers guide DAX generation using their prompts and visual metadata. A data agent does not necessarily return the original Power BI visual. Do not promise an identical chart or guaranteed latency improvement.

What to conclude: Prep data for AI is where the business writes down what the baseline agent had to guess: which tables matter, which answers are approved, and what the terms mean.

Facilitator cue: "We place each rule where it is consumed: business calculations in the model, routing and response behavior in the agent."

Step 4: Testing the optimized semantic model

Create and configure the AI-ready agent

1. Create a new data agent named `MfgOps_DA_AIReady_AB01`.
2. Add the `ManufacturingOpsAIReady` semantic model from the same workspace (use the **Refreshed** time to tell same-name models apart).
3. The Explorer lists **all 13 tables** of the model. Explicitly select only these eight: `Assets`, `Business Measures`, `Date`, `Inventory`, `Lines`, `Plants`, `ProductionLog`, and `Products`. Click **one checkbox at a time** and wait a second for each to save: rapid clicks can be lost when you leave the page.
4. Switch to another Explorer tab and back, expand the source, and confirm the eight checkboxes are still selected. An attached source with no selected tables cannot answer data questions (in an earlier test, this alone dropped the Lab 2 score from 3/3 to 1/3).
5. Open the agent's **Instructions** area and enter the following.

Scope

Answer manufacturing operations questions using the configured AI-ready semantic model: production, inventory, assets, plants, lines, operational performance, trends, and KPIs.

Audience and tone

Help plant managers, operations leaders, production supervisors, inventory planners, analysts, and executives. Be concise, professional, and clear. Explain manufacturing terminology when necessary.

Guidelines

- Decline sales, customer, vendor, purchasing, and purchase-order questions

with: "This question is out of scope for this agent. Please ask a manufacturing operations-related question."

- Respect the model's governed KPI definitions and time-period rules.
- When no period is supplied, use the latest 30 days of production data.
- State the requested start/end dates and the actual latest production date separately. If the requested end is later than the data-as-of date, explicitly disclose that records are only available through the data-as-of date. Do not imply full coverage through today.
- For YoY comparisons, always show both current and prior-year start/end dates.
- Clarify ambiguous plant, line, product, or date requests.
- State units, filters, and assumptions. Give the answer first, then supporting evidence.
- Return the requested aggregation grain. For a period broken down by line, request one full-period row per line, not a daily time series. Add daily granularity only when explicitly requested.
- Do not derive complete totals or averages from truncated detail. Requery at the requested aggregate grain.
- Do not invent explanations, missing values, or unsupported causal claims. Distinguish query failure from no matching records and from numeric zero.
- Location means plant location when a location grouping or filter is requested; do not add a location grouping otherwise.

Common terminology

- Machine means equipment manufacturer in this workshop. For by-machine questions request manufacturer-only grouping, not individual assets or lines. Individual equipment means asset name.
- Day production yield / DPY means the governed yield excluding Night shift, evaluated over the requested period. It does not mean daily yield.
- RQX: scrap rate.
- OEE: Overall Equipment Effectiveness; workshop alias "reliability".
- DPMP: defects per million parts.
- YoY / YOY: year over year. MTD: month to date. MOM: month over month.
- TP: Pumps and Turbines product categories.
- How often (inventory below reorder): count with the governed Inventory Risk SKU Count, i.e. product-plant-date snapshots below reorder quantity, not distinct days. Products below reorder means products with at least one such snapshot in the period, not only at the latest date. State the unit.

Period precedence and missing data

Explicit user dates or durations ALWAYS override the default. Apply the latest-30-day default ONLY when no period is specified. Last six weeks means 42 calendar days ending on the latest production date, inclusive; never replace six weeks with 30 days.

Never convert a blank or unavailable KPI into zero, including for a chart.

6. Close/save the instructions pane, reopen it, and confirm the text persisted.

7. Run the production data-as-of check for this model. Record its dates (reference: 1 June 2024 to **8 August 2026**).

8. Clear chat before each independent question below.

Why two layers of instructions? Model instructions alone do not fix every question: the agent's orchestrator first rephrases your question (for example into a daily series, an asset grouping, or "distinct dates currently below reorder") and only then generates DAX under the model rules. In the first rehearsal this broke the day-yield, machine and "this year" questions; in the third, the "how often" question (Question 7). Agent instructions control that rephrasing; model instructions control the DAX.

Question 1: Default period

What is our scrap rate?

Observation: check that the agent uses [Scrap Rate %] with the default period (30 days including the latest production date) and states the dates.

Expected result: 2.36% for 10 July to 8 August 2026, with data available through 8 August, in about 25 seconds (identical in all three rehearsals; some runs also list scrap units and production quantity).

What to conclude: The documented default (latest 30 days of data) is applied and disclosed. Compare with the baseline: same measure, but now the period is explicit and agreed.

If different: check the model's saved instructions, selected tables, and DAX filter. Retry with explicit start/end dates. Do not report success merely because the response repeats "30 days."

Question 2: Explicit period overrides the default

What is the OEE this year?

Observation: check that the agent uses the governed [OEE %], not an ad hoc formula; that "this year" overrides the 30-day default; and that the answer says it is year-to-date and where the data stops.

Expected result: 87.18%, 1 January to 8 August 2026, with "data available through 8 August 2026" and a note that it does not cover later dates (identical in all three rehearsals). Without the data-as-of guidance, the first attempt in rehearsal 1 implied coverage through today.

What to conclude: The governed [OEE %] replaces the improvised formula, and the answer is honest about the data stopping on 8 August. "This year" is answered as year-to-date *with* its real end date.

If different: ask for OEE for an explicit year and end date within coverage. Historical sample data does not become current-year data just because the question says "this year."

Question 3: Day production yield

What is our day production yield for the last six weeks? Break it down by lines.

Observation: check [Day Yield Pct] (Night shift excluded), one value per line, and the six-week boundaries ("six weeks" could mean 42 rolling days or six calendar weeks).

Expected result: eight lines, one value each for 28 June to 8 August 2026 (42 days): Line A1 97.56%, A2 97.61%, B1 97.63%, B2 97.66%, C1 97.61%, C2 97.43%, D1 97.56%, D2 97.56% (third rehearsal; earlier runs within 97.4–97.7%). One run grouped the lines under their plant and noted that blank line-plant combinations are not zero. Earlier instruction versions produced 30 days, or a 336-row daily series that hit the 200-row limit; the period-precedence and grain rules fix both.

What to conclude: Getting this right needed rules in both places: the model (which measure, which dates) and the agent (do not turn it into a daily series). Explicit durations must override defaults, and results must be computed at the grain the user asked for.

If different: request 42 days ending on a stated production date. Confirm the measure and line grouping in DAX rather than trusting the answer's title.

Question 4: Out-of-scope request

What were the TP sales last week?

Observation: a refusal, not a sales total. Recognizing TP does not make sales part of this agent's scope.

Expected result: the configured message "This question is out of scope for this agent. Please ask a manufacturing operations-related question.", without running a query (response time about 2 seconds).

What to conclude: Scope instructions control what the agent *tries* to answer. They are not security: users with model access can still query sales elsewhere. Use permissions or row/object-level security for real restrictions.

If different: check that you are in the AI-ready operations-only agent, not the baseline or multi-source agent. Check scope instructions and the focused schema. Never use scope prompts as a substitute for data security.

Question 5: Multiple abbreviations and year-over-year logic

What's the YOY TP reliability?

Observation: three terms must resolve: TP (Pumps and Turbines), reliability (governed OEE) and YoY (two comparable periods). Asking which periods you mean is acceptable.

Expected result: governed OEE for Pumps and Turbines only, with both periods and the data-as-of date stated. The **format varied** between rehearsals:

- a single comparison of the latest 30 days with the same window a year earlier: **86.83% (10 Jul – 8 Aug 2026) vs 86.76% (10 Jul – 8 Aug 2025), +0.07 pp** (rehearsals 2 and 3, three runs; one called the window "30 production days" although it is 30 calendar days); or
- a **monthly table for the last 12 complete months** (August 2025 – July 2026), each month against the same month a year earlier (for example July 2026: 85.53% vs 87.16%, –1.63 pp).

Both are valid readings of "YoY". In one run, one row's difference was miscalculated (86.07% vs 85.94% shown as "+0.00 pts" instead of +0.13). Check the arithmetic on a row or two.

What to conclude: Three business terms (YoY, TP, reliability) in one short question are resolved correctly because each is documented. "YoY" itself is still ambiguous (which periods?), so a comparison is only trustworthy when both periods are stated, and the numbers still deserve a quick check.

If different: expand the run steps. A DAX execution error is not evidence that prior-year data is missing. First establish coverage for both periods. Then use this diagnostic follow-up:

First establish the available production dates for Pumps and Turbines.
Ask me to choose comparable current and prior-year periods within coverage.
Then return [OEE %] for each period separately and the difference in percentage points. Do not interpret a failed query as missing data.

If necessary, ask the two period questions separately. Compare results only after both queries succeed; do not calculate a YoY percentage from a missing or zero denominator.

Question 6: Machine/manufacture interpretation

Show me the distribution of scrap rate by machine.

Observation: check that the query groups by Lines[Manufacturer] (the saved rule and the verified answer), not by asset or line.

Expected result: six manufacturers for 10 July to 8 August 2026: Fluke 2.60%, GE 2.41%, Mazak 2.38%, NI 2.36%, Marsilli 2.28%, Siemens 2.22%, with a six-bar chart (identical in all three rehearsals). If you see the same rate for every manufacturer, the query grouped by an Assets column, which does not filter production.

What to conclude: Instructions must match how the model is actually built. The original lab pointed this question at Assets[Manufacturer]; that column sits behind an inactive relationship and silently returns the same value everywhere. Check that a breakdown really varies.

If different: inspect conflicting metadata and ask explicitly:

Show [Scrap Rate %] grouped by Lines[Manufacturer] for the latest 30 days of production data. State the dates. I mean equipment manufacturers, not individual assets or production lines.

Different equivalent-looking columns are not automatically interchangeable: check relationships, grain, and totals before accepting the result.

Question 7: Frequency with a defined unit

Clear chat and ask the Step 1 question again, now on the AI-ready agent:

Which products have inventory below the reorder quantity? How often does that happen?

Observation: check that the answer uses Inventory Risk SKU Count, states the unit (product-plant-date snapshots), lists every product with at least one such snapshot, and states the period.

Expected result: the governed measure, with the unit stated, for a 30-day window. In rehearsal 3:

- **With the model instruction only** (before the agent line was added), the instruction was ignored twice: the orchestrator rephrased the question as "products *currently* below reorder" and "distinct dates", returned only AquaFlow 100 Centrifugal Pump (170 on hand vs 225 reorder) and counted 3 distinct days in 30 days, then 24 days in 12 months.
- **With the agent terminology line**, three runs out of three used the governed measure and stated the unit. Two used 10 July – 8 August 2026: **13 products, 23 snapshots** (FlowGuard 10 Control Valve highest with 3). One anchored the window on the last *inventory* date instead: 5 July – 3 August 2026, **15 products, 33 snapshots** (FlowGuard 10: 5), split by plant. Both match a direct DAX check for their window.

What to conclude: A business definition written once in the model was not enough on its own: the agent had already rephrased the question before the model rule applied. With the definition in both layers, the unit is now stable. What still varies is the anchor date, because inventory snapshots end on 3 August and production on 8 August: read the stated period, and specify the dates when that matters.

Facilitator cue: "The improvement is clearer, inspectable behavior. We still verify the actual query; instructions are guidance, not a guarantee."

Data agent runtime

Every data agent runs on a runtime: the orchestration, planning and routing logic plus the built-in tools that turn questions into queries. The **Standard** runtime (default, generally available) is described in the selector as "more consistent behavior with less frequent updates"; the **Preview** runtime as "latest improvements with frequent updates", including advanced DAX generation for semantic models. The runtime decides how and when changes to these core components reach your agent; it does not decide which data sources you can add. Preview is not a promise that every query will be faster or error-free.

1. In the AI-ready agent's **Runtime** selector, keep **Standard**.

2. Clear chat and ask the following original comparison question.

For each plant, return the smallest ordered set of line-and-shift combinations that reaches at least 80% of that plant's downtime. Include the first combination that crosses 80%. Show plant, line, shift, downtime minutes, rank within plant, share of plant downtime, cumulative share, and cumulative share before the row. Show the result as a table.

3. Record success/failure, total elapsed time, period, returned rows, and DAX.

4. Switch to **Preview**, clear chat, and repeat the identical question.

5. Check correctness before comparing speed. Within each plant, the rows must be ordered by descending downtime, use that plant's full downtime total, and stop at the first row whose cumulative share reaches or exceeds 80%.

6. Inspect tie handling. If downtime values tie, repeat both runtime tests with the same added instruction: Break ties by line name, then shift name, in ascending order. Keep the time range identical.
7. For a fuller comparison, perform ten independent runs per runtime with unchanged configuration and data. Alternate runtimes where practical.
8. **Stay on Preview** for the Code Interpreter exercise. Switching back to Standard shows "Switch to Standard and stop using some setup? Some setup only works on the Preview runtime": Code Interpreter is one of those.

Observation: two sets of measured outcomes, not a prescribed performance percentage. Record unsuccessful runs separately; report both failure rate and the median latency of successful runs. Do not silently drop failures or call a single faster response a benchmark. The original lab expected Preview to be about 45% faster by median latency; measure it rather than assume it. Preview also uses different internal tools (visible in the run steps).

Expected result (F16 capacity):

Runtime	Rehearsal 1 (10 runs)	Rehearsal 2 (1 run)	Rehearsal 3 (1 run)
Standard	5 of 10 executed, 0 fully correct, median 83 s	failed ("There's content here I can't work with")	query returned no rows; the answer said the model has no matching data (84 s)
Preview	10 of 10 executed, 5 fully correct, median 29.5 s	correct, 40 s	correct, 44 s

Preview was faster and more reliable on this question, but **neither runtime was error-free**. Expect Standard to fail on this question, and note how: in rehearsal 3 it turned an empty query result into "no data exists", the claim the model instructions forbid.

The period chosen can differ, so check it first. Correct values from a direct DAX check (each plant needs 9 of its 12 line-shift combinations):

Period used	Riverside: 9 rows / plant total	Rheinland: 9 rows / plant total
All history (1 Jun 2024 – 8 Aug 2026)	99,534 / 123,774 min (80.42%)	99,749 / 123,745 min (80.61%)
Latest 30 days (10 Jul – 8 Aug 2026)	3,897 / 4,679 min (83.29%)	3,303 / 4,118 min (80.21%)

Ten runs per runtime take about 30 minutes; in a timed workshop, run one or two each and compare with these tables.

What to conclude: Judge a runtime on correctness first, then speed. Preview was clearly better on this hard question, but neither runtime is guaranteed: complex questions need verification whichever you choose.

If different: if a runtime is unavailable, record that limitation. If a query fails, inspect its error and retry a simpler ranking before returning to the cumulative-threshold question.

Facilitator cue: "We compare correctness and speed together. A faster incorrect answer is not an improvement."

AI-assisted modeling changes

Preparing a large model for AI by hand is slow. Modeling Copilot can rename objects, add measures and columns, and write descriptions, if a human scopes and checks the work. This exercise modifies the **baseline** model. Perform it after the baseline question tests. In a shared workshop, the facilitator should coordinate who applies changes. A resumed, already-edited model will not reproduce the same starting behavior as a clean deployment.

1. Open the ManufacturingOps semantic model, not ManufacturingOpsAIReady.
2. Switch from **Viewing** to **Editing** mode. The first time, a banner says the model was converted to the large semantic model storage format and that version history is now available.
3. Select **Copilot** in the ribbon. This model has several columns called "name" or with cryptic names. Ask for proposed names without applying changes:

Identify all the name columns in this model. Using the table context and sample values, propose more descriptive names that clearly reflect the business meaning. Return a table with: Current Name, Proposed Name.

4. Copilot first asks "**Allow Copilot to make changes during this chat session?**" (with a **View impact** link). Review the scope and select **Allow** only if you intend to apply changes in this session; the choice lasts until you close the pane.
5. Review the proposed names with the facilitator. Before any modeling change, check dependencies in measures, relationships, reports, and verified answers.
6. If the proposal includes columns that are not entity names, narrow it:

Limit the proposal to columns that hold the name of an entity (customer, product, vendor, plant, line, asset and similar). Do not rename IDs, codes, dates, numeric fields or descriptive attributes.

7. Once approved, ask:

Approved, make the changes.

8. Reopen the affected tables to confirm the names actually changed. Open the baseline report and check affected visuals. An assistant's completion message is not sufficient evidence that all changes were applied.

Observation: proposal size varies, and Copilot's own summary of the impact can be wrong. Check what was saved, and whether measures still work.

Expected result:

- The first proposal took about 75 seconds and listed **43** rows in rehearsals 1 and 3 (including segment, Region, country, type, Mfr, Shift, Date[Quarter]), and **12** in rehearsal 2.
- The narrowing prompt returned **15** entity-name columns (for example Customers[custName] → Customer Name, Lines[line_name] → Production Line Name, Sales[Prod] → Product Name, Products[Name] → Product Name).
- After approval, all 15 renames were saved (11 of 12 in rehearsal 2). Copilot then warned that "any existing DAX expressions that referenced the old column names will now break". That is **not** what happened: the model updated the measure references itself, and every measure still returned values. The baseline report uses none of the renamed columns, so its visuals are unaffected.

What to conclude: Copilot speeds up modeling work, but its proposal is not deterministic (43 or 12 renames for the same prompt) and its description of the impact can be wrong in either direction. A human scopes the change, approves it and checks what was actually saved.

Optional: Add descriptions

Skip this if short on time. Ask:

This semantic model will be used by a Fabric data agent to answer manufacturing operations questions. Add descriptions based on the following guidelines.

DO: Add a description to every visible table, column, and measure. Keep descriptions concise and front-load the key meaning within the first 200 characters. Front-load preferred usage, disambiguation, and units. Make implicit knowledge explicit. Do not restate the field name or add DAX logic. Sample values if necessary to learn the domain and context. Include expected grain where useful. For calculation groups, describe the items and their use.

DON'T: Generate descriptions purely from AI without business context. AI-only descriptions tend to restate what the model structure already shows. Always validate descriptions with the user or a domain expert. Do not contradict descriptions across related fields.

If Copilot returns proposals for review, review them before saying:

Update the descriptions.

Observation: Copilot works for several minutes. Check whether it waits for approval or applies directly, then open a few objects, including the five demo measures in the **Ambiguous Names Demo** folder (`sls_amt_x`, `gm2_pct`, `po_ok_flagish`, `prd_yld_day`, `inv_rsk_u`), and compare each description with the DAX. The 200-character target is a concise-writing convention, not a hard limit of every product surface.

Expected result: two behaviors were observed:

- **Proposes, then waits** (rehearsal 1): it flagged **27** objects as uncertain. Apply only the confident ones (109):

Apply only the descriptions you are confident about. Skip every item you flagged for review.

- **Applies directly**, without asking (rehearsals 2 and 3; about 2 minutes). In rehearsal 2 its summary claimed the five demo measures were described, but **none was saved**. In rehearsal 3, 136 of the 137 visible objects were described and all five demo measures were saved, with meanings matching the DAX (for example `prd_yld_day`: "yield for non-night shifts, Day and Swing, only"). The description of `po_ok_flagish` faithfully repeats its formula but does not flag that the numerator counts all on-time orders while the denominator counts only orders of 100,000 or more.

If descriptions were applied without review, you can undo them through the model's version history.

What to conclude: AI-written descriptions are a draft, and the assistant's summary of what it changed can be wrong. Check what was saved, apply only what is correct, and leave uncertain items, and questionable formulas, for a domain expert.

Finish with this checklist for preparing a model for AI. Do not introduce new relationships or security changes solely to complete it:

- Business-friendly names for all visible tables, columns, and measures
- Concise, front-loaded descriptions on every visible object
- Synonyms for terms and abbreviations business users type
- Hierarchies where users naturally drill, such as Plant > Line > Machine
- A star schema with one-directional relationships where possible
- RLS/CLS applied and tested for restricted data

- An AI data schema scoped to the agent's topic or domain
- Verified answers for recurring, high-value questions
- AI instructions for calculations the agent would otherwise have to guess

Code Interpreter

Code Interpreter gives the agent a sandboxed Python environment to analyse the data it retrieves: calculations, statistics and charts the semantic model cannot produce. You can review the generated code, outputs and images in the run steps.

1. Return to your **AI-ready agent**, still on the **Preview** runtime.
2. Select **Add tools > Code Interpreter**, then **Add to data agent** in the "Add code interpreter?" dialog. Open the **Tools** tab and check that Code Interpreter is listed: without the confirmation, the tool is not enabled.
3. Clear chat and ask:

Show me a pivot table of products vs. the last six months by reliability. Put products on rows and months in columns.

4. Check the OEE measure, six-month boundaries, and coverage. Keep this conversation open and ask:

Show me a heatmap.

5. Expand the analysis steps. Inspect the Python code, input data, and output.

Observation: the heatmap should use the same product/month figures as the pivot. Missing cells should remain distinguishable from zero. Inspect how percentages and the colour scale are represented.

Expected result:

- Pivot question (about 1.5 minutes, 2 steps: query, then Python): governed OEE for 20 products × **March to August 2026**, with data available through 8 August. **August is partial** (1–8 August); it was labelled "Aug 2026 MTD", "August" or "Aug 2026*" with a footnote.
- Heatmap (about 1 minute): a products-by-months **image** with download links for the PNG and the CSV, and a Python step. In rehearsal 3 the agent re-ran the same model query before plotting (same values), rather than reusing the previous table.
- **Without the tool enabled** (an earlier rehearsal), the agent still answered "Show me a heatmap" with an emoji-coloured text table. It looks like a heatmap, but no Python ran.

What to conclude: Code Interpreter adds analysis and charts the semantic model cannot produce, on the same governed numbers. Check that Python really ran (an image and a code step, not coloured text), and check partial periods: a month labelled "August" may contain only 8 days.

If different: if no Python tool ran, do not present the result as a Code Interpreter demonstration. Confirm the tool is enabled and the runtime is Preview, then ask: Use Code Interpreter to render the preceding table as a heatmap. Preserve missing cells and label the OEE percentage scale.

Optional: Detrending and FFT

This kind of analysis is common on manufacturing time series: remove the trend from a signal, then use a Fast Fourier Transform to look for repeating patterns (cycles) that are hard to see otherwise. Clear chat and ask:

Detrend the daily scrap rate for Pump Impeller for the last two months, run FFT on it, and identify any dominant modes.

Observation: check the product, daily measure, period, number of observations, how days without production were handled, the detrending method, and the frequency units.

Expected result: two steps (query, then Python), with the data-as-of date stated. The **method and the conclusion differed** in each rehearsal:

- Rehearsal 1: June–July 2026, days without production **interpolated**, linear trend removed, a dominant cycle of about **15 days**.
- Rehearsal 2: 9 June – 8 August 2026 (61 calendar days, 31 with data), **no interpolation**, strongest mode about **7.75 production days**, with a warning that a calendar-day cycle cannot be inferred.
- Rehearsal 3: 1 July – 8 August 2026 (20 observations, 19 dates excluded, not set to zero), three near-equal peaks (about 10, 6.7 and 5 observed days) and the conclusion **"no reliable periodic mode"**.

What to conclude: Python makes advanced statistics easy to request, but every method rests on assumptions, and the agent picks them for you: here, three different periods, gap treatments and conclusions for the same question. Read how missing days were treated before trusting any pattern, and treat a peak as a lead to investigate, not a root cause.

If different: an empty or irregular series may not support the requested analysis. Request coverage and sampling checks before accepting any result. Do not silently fill gaps with zero. A spectral peak is a candidate pattern, not proof of a manufacturing root cause.

Facilitator cue: "Python extends what we can calculate, but it does not remove the need to check the data and the statistical assumptions."

Lab 2: Programmatic evaluation of data agents

Goal: learn the evaluation process, calibrate an LLM judge against human labels, evaluate your operations-only AI-ready agent with the Fabric data agent Python SDK, and inspect the results in MLflow.

The SDK gives code-first access to data agents: create, configure, update, publish and evaluate them without the portal. It runs in a Fabric notebook or, after authentication, in your own environment, but the **Responses API evaluation only works inside a Fabric notebook**. Run both notebooks **inside Fabric**. Allow extra time for environment startup, package installation, and service calls.

Step 1: LLM-as-Judge calibration

1. Open the public [judge calibration workbook](#) for review.
2. Inspect its two sheets. `calibration_development` was labeled by the agent creator to develop and refine the judge rubric; `calibration_holdout` is kept sealed and labeled independently to validate the final judge before approval. The labels are provided for this lab; in practice, label with other developers and end users.
3. Open `JudgeCalibration` in your Fabric workspace.
4. Inspect its **Configuration** cell. Keep the supplied model, rubric, data reference, and acceptance thresholds unless the facilitator instructs otherwise. Note the judge model and `JUDGE_REGISTRY_EXPERIMENT`.
5. Select **Run all**, and wait for execution to finish. If package installation requests a session restart, follow the notebook instruction before continuing.
6. Review the development/holdout results and the notebook's registration decision. Do not register a failed judge by lowering thresholds to make the exercise pass.
7. Open the MLflow run linked by the notebook (experiment `mfg-ops-judge-registry`). Inspect the rubric, model, human-label agreement metrics, data reference, and registration tags.

Observation: the notebook creates or reuses the evaluation Lakehouse, loads the labeled workbook, evaluates the candidate judge, logs evidence, and registers a champion **only if its acceptance rules are met**.

Expected result: champion judge registered (model `gpt-5.1`, 34 development and 18 holdout examples) in about 2 minutes. Agreement with the human labels was **97.1%** (development) and **94.4%** (holdout, Cohen's kappa **0.886**) in rehearsal 1, and **100%** on every metric in rehearsals 2 and 3. If you run the notebook as a scheduled job, its status can show **Completed** or **Cancelled** (the last cell stops the session on purpose); check the MLflow run and its `judge_status = champion` tag instead.

What to conclude: Before an AI grades another AI, check it against human judgement. 94–100% agreement on unseen examples justifies using this judge for automated tests; a judge below the threshold would not be registered. Step 2 shows that this is necessary, not sufficient.

If different: distinguish authentication, model availability, capacity, package, or storage errors from poor agreement with human labels. Stop before agent evaluation if no approved judge is available.

Facilitator cue: "Before asking an AI to grade another AI, we check that its grading agrees with the human standard."

Step 2: Evaluate your AI-ready agent

1. Open `EvaluateDataAgent`.
2. Find **Select the Data Agent**. Set the following to the **exact display name of your own agent**, not the illustrative `..._SAP` default.

DATA_AGENT_NAME = "MfgOps_DA_AIReady_AB01"

3. In **Configuration**, confirm GT_MODEL = "ManufacturingOpsAIReady". Leave REFRESH = "No" unless you intentionally need a refresh. Refreshing during a comparison can change the answer key.
4. Confirm that the agent is the operations-only agent from Lab 1. The workshop evaluation includes a sales refusal check; it is not the same policy as the multi-source agent in Lab 3.
5. Review the evaluation workbook used by your notebook. The source version accompanying this guide uses eval_set_L400.xlsx (columns: id, goal, question, answer type, ground-truth DAX, expected behavior, expected measure, expected period, policy area) and selects three cases when TEST_MODE = True. Inspect TEST_CASE_IDS; do not assume the workbook itself contains only three rows.
6. Confirm that the eight source tables remain selected and the model and agent instructions are saved.
7. Select **Run all**. The SDK asks each question to the agent, generates the numerical ground truth from the semantic model, and the calibrated judge rates each answer. Run steps, configuration, answers and verdicts are logged to the mfg-ops-data-agent-eval MLflow experiment.
8. Review the overall score and the **Per-question review** under the results section. If accuracy is below 100%, read each failed row's **answer**, **expected answer** and **judge reason** before changing anything. Open the MLflow run.

Observation: inspectable per-question answers, DAX/run steps, ground truth or policy expectations, judge reasoning, and metrics. Completion of the notebook is not equivalent to 100% answer accuracy, and a failed case is not automatically a wrong answer.

Expected result: about 3–5 minutes per run, 0 infrastructure errors, the judge loaded from the registry, and a score that **varied** while the agent's answers stayed correct:

Run	Score	What happened
Rehearsals 1 and 2	3/3	all answers correct and accepted
Rehearsal 3, run 1	2/3	"Which line has the highest scrap rate?" answered Line C2 – Sensor Calibration, 2.60% (= ground truth) but failed: "adds specific scrap units, production quantity, and date details that are not present in the expected answer"
Rehearsal 3, rerun	1/3	the same line answer, and the year-to-date day production yield 97.67% (= ground truth), both failed for adding "unsupported extra details" (the period and data-as-of date)
Earlier test, tables not selected	1/3	only the refusal passed

The refusal case ("Which product had the highest sales this year?") passed every time. The rubric fails answers that "fabricate facts" and tells the judge to use only the expected answer, so the period and data-as-of date that the agent instructions require can be read as unsupported facts.

What to conclude: Automated evaluation turns "it seems to work" into a repeatable score, and it catches configuration mistakes (one unselected setting dropped the score to 1/3 while the agent still looked fine in chat). But a calibrated judge is still a model: here it failed correct answers for being more transparent than the expected answer. Read the judge's reason before fixing the agent, and add such cases to the calibration set rather than removing useful detail from the answers.

If different: when a refusal succeeds but data questions fail, first compare the failed answer with the expected answer. If the value matches, it is a judge disagreement, not an agent error. Otherwise inspect source selection, data access, query errors, and the period used. A policy refusal can succeed without successful data retrieval.

Correct one issue at a time, label the next run using the notebook's STAGE field, and rerun. Preserve the first run for comparison. The supplied source evaluates the agent's draft (sandbox) configuration: do not assume this proves the published endpoint is identical.

Checkpoint: do not move on with an unexplained failed factual answer. Record the failed case and its cause, or ask the facilitator for help. A three-case evaluation is a learning exercise, not production certification.

Lab 3: Adding multiple data sources

Goal: add Lakehouse data for downtime reasons and product sales, inspect the resulting routing and queries, then publish and consume the agent.

So far each agent used one semantic model. A data agent can combine several OneLake sources: Lakehouse, Warehouse, SQL database, KQL database, Azure AI Search and ontologies (see the Fabric data agent documentation for the current list). In this lab you use the Python SDK to add a Lakehouse to a copy of your AI-ready agent.

This explicitly broadens scope: **product sales become allowed through OpsRefData**. Customer-level data, vendors, and purchasing remain outside scope.

Step 1: Build the Lakehouse source

1. Open `BuildOpsRefData`.
2. Use the supported Python 3.11 or 3.12 notebook runtime for the supplied notebook. Do not convert it to an unrelated Spark runtime solely for this exercise.
3. In a shared workshop, ask the facilitator whether `OpsRefData` is already prepared. The notebook rebuilds shared data and SQL objects; participants should not all rebuild it concurrently.
4. If it needs building, select **Run all** and wait for the final result.
5. Return to the workspace and open the `OpsRefData` Lakehouse.
6. Switch to its **SQL analytics endpoint**.
7. Expand the `fda` schema and confirm the `Downtime_Reasons` and `Sales_Orders` views, plus the `Downtime_By_Line_Type` table-valued function.
8. Open **New SQL query** in that endpoint and run:

```
SELECT 'Downtime_Reasons' AS [Object], COUNT(*) AS [Rows]
FROM [fda].[Downtime_Reasons]
UNION ALL
SELECT 'Sales_Orders', COUNT(*)
FROM [fda].[Sales_Orders];
```

```
SELECT *
FROM [fda].[Downtime_By_Line_Type]('Assembly')
ORDER BY [Downtime Minutes] DESC;
```

9. Record source coverage:

```
SELECT MIN([Date]) AS [First Date], MAX([Date]) AS [Latest Date]
FROM [fda].[Downtime_Reasons];
```

```
SELECT MIN([Order Date]) AS [First Sales Month],
       MAX([Order Date]) AS [Latest Sales Month]
FROM [fda].[Sales_Orders];
```

Observation: both views contain queryable data, and the function returns the Assembly downtime-reason breakdown. The function is all-history and has no date parameter; use the view for date-filtered analysis.

Expected result: the notebook finishes in about 2 minutes. `Downtime_Reasons` 75,490 rows and `Sales_Orders` 1,160 rows. The `Assembly` function returns Equipment Failure 72,110, Changeover 31,949, Planned Maintenance 20,752, Material Shortage 19,160 and Operator Error 15,608 minutes (identical in all three rehearsals).

Important — dates differ from the models: BuildOpsRefData generates Lakehouse data **up to the day before you run it**, not up to the models' dates. In the rehearsals, run on 3 and 4 October 2026, downtime reasons ran from 1 June 2024 to 3 October 2026 and sales months to 1 October 2026. The semantic models stop on 6 July / 8 August 2026. So "latest 30 days" can mean a different period in each source. Keep this in mind for Step 3.

What to conclude: Different sources rarely have the same data freshness. Before combining them, know each one's dates.

If different: SQL endpoint metadata can lag table creation. Inspect the notebook's first failed cell, wait for endpoint metadata refresh, and rerun the affected step under the facilitator's guidance. Do not treat missing SQL objects as successfully configured sources.

Important grain distinction: Sales_Orders contains monthly product/plant facts; Order Date is the first day of the sales month. It is not a daily transaction feed. A latest month key does not prove that the month is complete.

Step 2: Create the multi-source agent with the SDK

1. Open CreateMultiSourceDataAgent in the **same workspace** as your AI-ready agent and OpsRefData.

2. Inspect **Step 1 - Parameters** and update:

```
BASE_DATA_AGENT_NAME = "MfgOps_DA_AIReady_AB01"
MULTI_SOURCE_AGENT_NAME = f"{BASE_DATA_AGENT_NAME}_MultiSource"
SEMANTIC_MODEL_NAME = "ManufacturingOpsAIReady"
LAKEHOUSE_NAME = "OpsRefData"
PUBLISH_CHANGES = True
```

3. Before running, review the notebook's publication behavior and the target name. PUBLISH_CHANGES = True publishes in Step 9. If you are not ready to publish, set it to False and stop before the published MCP test in Step 10.

4. Run cells from **Step 0** through **Step 8** in order. Allow package installation and source discovery to finish.

5. Read the output: the base agent must exist; the eight model tables and three Lakehouse objects must be selected.

6. Open the newly created ..._MultiSource agent in Fabric and inspect its sources, agent instructions, and Lakehouse **Setup** guidance/examples.

Observation: the notebook copies the base agent's model source, adds the Lakehouse with its views and function, writes new agent instructions for the new routing, adds a data source description, instructions and example queries, publishes the agent and tests it through MCP. Your base agent remains available and unchanged.

Expected result: about 4–5 minutes (in rehearsal 3 the job also waited 3 minutes in the capacity queue). The multi-source agent has the eight model tables, the three Lakehouse objects, about 2,800 characters of agent instructions, about 4,400 characters of Lakehouse instructions and **five** example queries. It came back on the **Standard** runtime and **without Code Interpreter**, although the base agent used both.

Important: the notebook is not a complete clone of every custom base setting, runtime, or tool. Recheck any customizations you need.

What to conclude: A code-first copy is repeatable and reviewable, but it only copies what it was written to copy. Check runtime and tools as well as sources and instructions.

The notebook broadens sales scope and replaces agent instructions. Do not keep the operations-only "all sales are out of scope" rule on this multi-source agent. Conversely, do not broaden the Lab 1 agent.

Inspect and clarify time handling

The supplied notebook contains snapshot-relative time defaults, including "this year" based on the selected source's latest date. Inspect those rules: they can differ from the calendar-relative meaning a user expects.

For the cross-source exercises, use **explicit matching periods**. When comparing daily production with monthly sales, use calendar months whose coverage is confirmed in both sources. Do not claim that simply taking the earlier maximum date proves complete coverage, or prorate a monthly sales row into daily sales.

Never join daily downtime detail directly to monthly sales detail. Aggregate each side independently to the same plant/month grain first. Keep missing groups visible when the question requires them, and do not infer causality from a correlation.

Step 3: Publish and test the MCP endpoint

1. Return to **Step 9 - Review and publish**. Review the staging source names and saved settings.
2. If publication is intended and authorized, run Step 9 with `PUBLISH_CHANGES = True`. Otherwise, publish your agent manually after reviewing it and only then proceed.
3. Run **Step 10 - Query the published agent through MCP**. Keep the notebook session open until you finish the later post-publication checks.
4. Inspect each test result, not just `is_error`. A successful MCP transport response can still contain a refusal, incomplete answer, or query failure.

The supplied tests cover the following:

Test	Question / intended route
Semantic model	"What was the scrap rate in the latest 30 days?" - governed model measure.
Lakehouse	"Which product had the highest sales?" - actual revenue from <code>Sales_Orders</code> .
Both sources	"Which line had the most downtime minutes, and what were its leading downtime reasons, using the same latest 30-day period?" - align periods and distinguish totals from reasons.
Function	"Across all available history, what is the downtime-reason breakdown for Assembly lines?" - the all-history function is appropriate.

Observation: a published endpoint can be initialized and queried using the signed-in identity. Read each answer against its analytical criteria. Keep authentication tokens out of screenshots, notebook exports, and public repositories.

Expected result: the cell completes and its `is_error` assertion passes. Reading the answers:

Test	What you will see
Semantic model	2.36% , 10 July to 8 August 2026 — same as the Lab 1 agent. Correct.

Test	What you will see
Lakehouse	The top product by revenue, but the period varies : HelioGen 3000 Steam Turbine, 165,376,000 "across all available order dates" (three runs), or HelioGen 2000 Gas Turbine (7,722,000) over the Lakehouse's own "latest 30 days" (two runs). Because sales are monthly rows dated on the 1st, that window contained a single month row, October 2026, dated 1 October: one day into the month. Both answers are correct for their period; check which period the answer states.
Both sources	Check it. Line A1 - Pump Assembly, 1,588 minutes, 10 July – 8 August 2026, reasons adding up to 1,588: this aligned answer came back in 6 of 6 runs in rehearsals 2 and 3. In rehearsal 1, the answer mixed periods : the total came from the model (ending 8 August) but the reasons from the Lakehouse's September–October data (adding up to 1,833), while claiming a single period. In every case <code>is_error</code> was <code>False</code> .
Function	All-history Assembly breakdown, identical to the SQL function above. Correct.

This is the key Lab 3 lesson: **a transport success is not an answer check**. For the combined question, check that the reasons add up to the line's total. To remove the risk, rerun it with explicit dates, in the agent's Test chat or by replacing that entry in `MCP_TEST_QUESTIONS`:

For 2026-07-10 through 2026-08-08 inclusive, which line had the most downtime minutes, and what were its leading downtime reasons in that same period?
Apply exactly these dates to both sources and state them.

Expected result: Line A1 - Pump Assembly, **1,588** downtime minutes;

reasons Equipment Failure 713, Changeover 322, Planned Maintenance 207, Material Shortage 190, Operator Error 156 (sum 1,588).

What to conclude: "No error" is not "correct". With relative periods, the agent *can* combine numbers from two different time windows and present them as one; it does not do so every time, which makes it harder to spot. With explicit dates, the two sources reconcile exactly (reasons add up to 1,588). For cross-source questions, fix the period yourself.

If different: check whether the agent was published, whether your identity has source access, and whether the endpoint has become available. A saved draft does not update the published endpoint. Do not change tenant permissions just to hide a failed test.

Step 4: Creator Assistant - Build agent with AI

Use this exercise on the **multi-source agent** and its Lakehouse source. **Build agent with AI** (the Creator Assistant) helps you generate and refine the configurations that drive the agent: agent instructions, data source instructions and descriptions, and example queries. It currently supports SQL and KQL sources, so it works on the Lakehouse here. It can propose and save changes, but you must verify each requested change was persisted.

1. Select **Build agent with AI**. The chat box now reads "Converse with me for improving your data agent configurations".
2. Enter the original multi-change request:

Can you update the agent instructions and the `OpsRefData` data source instructions, and add a few-shot example so that when a user asks about the `turbomachinery` category, the query filters for Pumps and Turbines, but not Motors? Please sample the values first to confirm.

3. Review the proposal. The assistant should query the Lakehouse to confirm the category values (Motors, Pumps, Sensors, Spare Parts, Turbines, Valves) before proposing instructions.

4. When satisfied, say:

Apply these changes.

5. The assistant writes **one artifact per turn** and asks for confirmation each time. Whenever it shows a draft and says *Reply "save"*, reply:

save

6. Inspect **three separate locations** before continuing:

Location	Required persisted content
Agent instructions	Turbomachinery means Pumps and Turbines, excluding Motors; sales routes to OpsRefData.
OpsRefData source instructions	SQL filters [Category] IN ('Pumps', 'Turbines') for turbomachinery.
OpsRefData example queries	A saved question/query pair demonstrates that category filter.

Observation: a single "Apply" does **not** write all three changes. The assistant writes **one artifact per turn**, asks for "save" or for which artifact to do next, and the exact sequence varies. Compare the saved examples with the ones the notebook created.

Expected result: all three locations contained the change at the end of every rehearsal, after 4 to 6 messages. Rehearsal 3:

You send	The assistant
The original request	Drafts complete agent instructions with a turbomachinery rule (about 1 minute) and asks you to reply "save". It did not show the sampled values.
Apply these changes.	Saves the agent instructions, then asks which artifact next: (A) source instructions or (B) example queries.
Now update the OpsRefData data source instructions with the turbomachinery rule.	Drafts the source instructions; asks for "save".
save	Saves them; offers (A) examples or (B) a test query.
Now add the turbomachinery few-shot example query to the OpsRefData source.	Drafts eight examples; asks for "save".
save	"Added 8 example queries ... backed up the previous examples."

In rehearsal 2, the second "save" was needed after "Apply" (nothing was saved by "Apply"), and the assistant drafted the source instructions on its own. In rehearsal 1, each change needed its own request.

Two side effects to check:

- **Existing examples were replaced.** In rehearsal 3 only one of the five notebook examples survived (the all-history Assembly breakdown). "Why was downtime highest for Assembly lines in the latest 30 days?", "Which product had the highest sales?", "What is revenue by product category this year?" and "Which plant has the highest sales margin this year?" were replaced, the last by a turbomachinery-only version. Rehearsal 2 also saw the existing examples rewritten.
- **The draft changes more than you asked.** The new agent instructions rewrote the out-of-scope message and added a mandatory line to every turbomachinery answer: "Applied turbomachinery filter: Category IN ('Pumps','Turbines') (Motors excluded)."

What to conclude: The assistant's "done" message is not proof. Check every place a change should land, and check what else changed: an assistant that rewrites a whole configuration can drop examples or wording you relied on.

If different: ask for the missing change individually, or use the source's **Setup** editor to add it. Preserve existing routing, grain, and date rules. For a simple category-only example, add the following question and SQL pair.

Across all available sales history, what are revenue and units sold for turbomachinery, excluding Motors?

```
SELECT SUM([Revenue]) AS [Revenue],
       SUM([Units Sold]) AS [Units Sold]
FROM [fda].[Sales_Orders]
WHERE [Category] IN ('Pumps', 'Turbines');
```

This example explicitly requests all available history. Do not reuse its unfiltered dates for questions asking for a particular month or year.

7. Ask the assistant:

Test this change and verify that it works as expected.

It runs its own test queries (about 1 minute). In rehearsals 2 and 3 it listed the categories in the data, then tested the **latest 30 days**: Turbines 16,818,000 / 87 units plus Pumps 1,336,600 / 180 units (total 18,154,600), which is only the October 2026 month row. Its numbers therefore do not match the all-history totals below. That is expected: compare like with like.

8. Return to the normal chat (select **Test data agent** in the toolbar; it may be under **More (...)** on a narrow window). Clear chat and ask the same all-history question.

9. Run the SQL above directly in the OpsRefData SQL analytics endpoint. Compare revenue and units, source, categories, and period.

10. Inspect a category breakdown if totals disagree:

```
SELECT [Category], SUM([Revenue]) AS [Revenue],
       SUM([Units Sold]) AS [Units Sold]
FROM [fda].[Sales_Orders]
GROUP BY [Category]
ORDER BY [Category];
```

Observation: the turbomachinery result must equal the Pumps plus Turbines totals for the same period and exclude Motors. Do not copy a numerical total from a different environment as the answer key.

Expected result: revenue **419,704,600** and **7,118** units, filter [Category] IN ('Pumps', 'Turbines'), identical in the direct SQL query, the Test chat (about 10 seconds, ending with the new "Applied turbomachinery filter" line), the published agent, the MCP endpoint and Microsoft 365 Copilot, in all three rehearsals. The category breakdown shows Motors at 33,728,200 / 9,026, correctly excluded. Your totals can differ if your Lakehouse was built on another date; what must match is the agent's answer and your direct SQL query.

What to conclude: The business term "turbomachinery" now consistently means Pumps + Turbines, and the agent's answer equals a direct database query: that is the test of a correct configuration.

Facilitator cue: "We asked for three changes, so we inspect three saved settings. Then we compare the agent's answer with a direct source query."

Step 5: Description and final publication

1. Ask Creator Assistant:

Generate a concise description of what this data agent does. Focus on the specific business domain and the questions it answers. Avoid generic, reusable descriptions. Make it specific to this data agent's unique purpose and capabilities. Do not use bullet points and do not mention table names, column names, datasets, schemas, or other technical implementation details. Use clear language that accurately describes its value.

2. Review and copy the description. It should describe both manufacturing operations and product sales without claiming unsupported functionality. The assistant also offers to "save" it; you can ignore that and paste it into the Publish dialog instead.

3. Select **Publish**. The **Description of purpose and capabilities** field is **prefilled**: in rehearsal 3 with the notebook's change note ("Added OpsRefData for downtime root causes and product sales; updated routing, ..."), not a capabilities description. Replace it with the description from step 1.

4. Append the original output-handling guidance. A description is required for the agent to work well in Microsoft 365 Copilot, and this sentence asks Copilot not to rewrite the agent's answer:

The output from the data agent should be delivered as-is, without summarizing, rephrasing, or adding extra interpretation or insight.

5. Publish the latest changes. Keep **Also publish to Microsoft 365 Copilot** off unless you are completing the next step and have the required authorization.

6. Return to the still-open notebook session and rerun the MCP query cell with the same turbomachinery question used in Step 4, replacing or adding an entry in MCP_TEST_QUESTIONS. Do not rerun the configuration-writing cells: they can overwrite the Creator Assistant changes.

7. Compare the published answer with the direct SQL and draft answer, using identical scope and dates. When finished, run the notebook's final session stop cell.

For step 6, replace only the MCP_TEST_QUESTIONS list in notebook Step 10 with the following, then rerun that Step 10 cell:

```
MCP_TEST_QUESTIONS = [
    (
        "turbomachinery",
        "Across all available sales history, what are revenue and units sold "
        "for turbomachinery, excluding Motors?",
    ),
]
```

Observation: the published agent must reflect the latest configuration. The description guidance can reduce Microsoft 365 Copilot rephrasing; it does not guarantee verbatim output.

Expected result: the generated description took under a minute and covered production performance, quality, downtime causes, inventory and product sales, including the turbomachinery mapping. After publishing, the MCP endpoint returned 419,704,600 / 7,118 with Motors excluded, `is_error = False`, and the published configuration contained the new examples.

What to conclude: Publishing creates the version others use. Testing the published version, not just the draft, confirms that what you validated is what users get.

Step 6: Consume the agent in Microsoft 365 Copilot

Complete this step when the workshop account has the required Microsoft 365 entitlement and Copilot extensibility enabled. Use the **same account and tenant** as Fabric. Publishing to the Agent Store is an additional action, not a prerequisite for the preceding Fabric and MCP exercises.

1. In your agent's **Publish** dialog, review the description and enable **Also publish to Microsoft 365 Copilot**.
2. Publish and open Microsoft 365 Copilot with the same account.
3. Open **Agents / Agent Store**, locate your agent by its exact name, and open it. Navigation labels can vary.
4. If it has not appeared, refresh the agent list or expand navigation and allow propagation time. If still unavailable, ask the facilitator to check entitlement and extensibility settings.
5. Start a new conversation directly with the agent, or select it using @.
6. Ask the **same** turbomachinery question used for SQL, draft, and MCP.
7. Compare revenue, units, categories, and period. Do not require identical prose; Microsoft 365 Copilot has its own orchestration layer.

Observation: the agent must be discoverable for the authorized user and return a source-grounded answer consistent with the equivalent direct query.

Expected result: the agent appeared in the Agent Store within 5 minutes (rehearsal 3, listed under your agents) to 20 minutes (rehearsals 1–2) of publishing; search for part of its name. The search result shows the full name with a generic description ("Declarative agent that uses Data Agent to answer questions"); the agent's details card cuts the name to 30 characters (MfgOps_DA_AIRReady_AB01_MultiSo). Select **Open**. The answer took 40–80 seconds: 419,704,600 revenue and 7,118 units, Pumps and Turbines only, with the "Applied turbomachinery filter" line and suggested follow-up questions.

What to conclude: The same governed answer reaches users in Microsoft 365 Copilot. The wording can differ because Copilot adds its own layer; compare the numbers and filters, not the prose.

If different: confirm agent identity, account, tenant, publication state, underlying source access, and time range. A missing Agent Store entry is not a failed DAX or SQL calculation.

Do not share the agent externally as part of this exercise. Agent Store publication does not grant recipients access to underlying sources. Microsoft 365 consumption can process results under Microsoft 365's data handling terms and outside Fabric's compliance boundary; follow your organization's policy before enabling it.

What you learned

- **Lab 1:** built a data agent on a semantic model; read the paraphrase, DAX and answer behind each response; saw where it guesses (periods, KPIs, jargon, grain, business terms); fixed that with an AI data schema, verified answers, model and agent instructions; used Copilot to rename and describe model objects; compared the Standard and Preview runtimes; used Code Interpreter for pivots, charts and statistics.
- **Lab 2:** calibrated an LLM judge against human labels, ran an automated evaluation with the Python SDK, and learned to read a failed case before acting on it.
- **Lab 3:** added a Lakehouse with the SDK, aligned periods across sources, refined the agent with Build agent with AI, published it, and checked the same answer through the MCP endpoint and Microsoft 365 Copilot.

Workshop completion checklist

Area	Evidence to retain in your own approved environment
Baseline agent	Selected source tables, data-as-of dates, and inspected responses.
AI-ready agent	Eight selected tables, consistent model/agent guidance, governed KPI queries, and correct sales refusal.
Relative questions	Declared reference dates and exact resolved periods; no silent substitution of missing months.
Runtime	Recorded correctness and timing; unsuccessful runs remain visible.
Modeling	Approved names/descriptions persisted, with dependent content checked.
Code Interpreter	Python execution and chart inputs inspected, or feature blocker recorded.
Evaluation	Approved judge plus per-question agent results and MLflow evidence.
Multi-source	Both sources selected, routing inspected, source grain and periods understood.
Creator Assistant	All three requested configuration changes persisted and directly checked.
Publication	Matching question/scope checked in draft and published MCP.
Microsoft 365 Copilot	Matching factual result, or a clearly recorded access/feature blocker.

You have completed the learning path when you can explain both successful answers and unresolved failures. Do not mark an unexecuted optional feature as passed.

Troubleshooting

Symptom	First action
Setup notebook fails with HTTP 403 at the DataFolder parameter step	Fix that cell's Authorization header to "Bearer " + power_bi_token, then rerun (see Before you start).
Agent loses table selections	Click one checkbox at a time, wait for each to save, then reopen the Explorer to confirm.
Every manufacturer shows the same scrap rate	The query grouped by an Assets column (inactive relationship). Use Lines[Manufacturer].
Six-week question uses 30 days, or returns a daily series	Check the period-precedence and grain rules in both model and agent instructions.
Combined downtime question mixes periods	Expected with relative periods: the Lakehouse runs to today, the models do not. Ask with explicit dates.
Stuck in Build agent with AI mode	Select Test data agent in the toolbar (or under More (...)).
"You can only open 20 items at a time" when opening an item	Close some open items in the left navigation bar (each opened item stays there), then retry.
"Show me a heatmap" returns coloured text, not an image	Code Interpreter is not enabled: add it again and confirm Add to data agent .
Creator Assistant shows a draft but nothing changes	Reply save: it writes one artifact per turn and waits for confirmation.
Agent not found in Microsoft 365 Copilot	Wait 5–20 minutes after publishing and search part of the name.
Clear chat seems to do nothing	Confirm the "Clear chat?" dialog (or tick Don't show this again).
Two models with the same name in the OneLake catalog	The catalog has no workspace column: use the Refreshed time or the filter.
Code Interpreter does not run	It needs the Preview runtime and the Add to data agent confirmation.
Evaluation fails an answer whose value is correct	Read the judge reason: it can penalize extra details such as the period. Treat it as a judge disagreement, not an agent error.
Example queries disappeared after Creator Assistant	It can replace the whole example set. Re-add the ones you need in Setup .
Standard runtime answers "no data" to a complex question	An empty query result is not missing data. Retry on Preview or simplify the question.
"Last month" returns an unexpected month	Inspect the DAX reference date and the fact table's coverage; ask with explicit dates.
Baseline and AI-ready totals differ	Align measure, filters, date range, and underlying data coverage before comparing quality.

Symptom	First action
AI-ready agent cannot answer data questions	Confirm that tables are selected, the source is accessible, and the query executed.
RQX gives quality rather than scrap	Remove the conflicting model instruction; confirm the query uses [Scrap Rate %].
Day yield includes Night shift	Inspect [Day Yield Pct] selection and model-level guidance.
YoY answer says "no data" after a failed query	Treat it as a query failure; establish coverage with a successful independent query.
Notebook cannot find an agent	Replace the sample name with the exact agent display name in that workspace.
Notebook evaluation succeeds but score is low	Read each failed case and its ground truth; notebook execution is not answer correctness.
Creator Assistant says done, but behavior is unchanged	Inspect agent instructions, source instructions, and few-shot examples separately.
Sales behavior changes after Lab 3	Expected only on the multi-source agent, which routes product sales to OpsRefData.
SQL objects are absent after building the Lakehouse	Inspect the failed build/metadata step; wait for endpoint synchronization before retrying.
Published result differs from draft	Republish the reviewed configuration and use the same question, source, and period.
Copilot wording differs from Fabric	Compare facts and filters; the extra orchestrator can rephrase results.

Sources and license

Based on the MIT-licensed [Microsoft Fabric Data Agent Workshop](#). Lab and notebook reference: [commit 136bcf196a7f935fb030bd9fe81ad0e133d2fbf3](#). Check your deployed notebook cells if their version differs.

Product guidance:

- [Semantic model best practices for data agents](#)
- [Data agent configurations](#)
- [Data agent query-generation best practices](#)
- [Consume a data agent from Microsoft 365 Copilot](#)

This is a community workshop guide, not a product warranty or a guarantee of deterministic AI responses. Microsoft product names remain their owners' trademarks. This guide does not imply Microsoft sponsorship.

MIT license

Copyright (c) Microsoft Corporation.

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.